

Recommend.R

shariqueshafi

2025-11-26

```
#Knowledge based, Collaborative, Content Based  
#Content based - easiest to create , makes similarity matrices.  
#KBRs/ Apriori Algorithm/ Market basket Analysis - Is done on transactions  
library(arules)
```

```
## Loading required package: Matrix
```

```
##  
## Attaching package: 'arules'
```

```
## The following objects are masked from 'package:base':  
##  
##      abbreviate, write
```

```
library(arulesViz)  
library(dplyr)
```

```
##  
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:arules':  
##  
##      intersect, recode, setdiff, setequal, union
```

```
## The following objects are masked from 'package:stats':  
##  
##      filter, lag
```

```
## The following objects are masked from 'package:base':  
##  
##      intersect, setdiff, setequal, union
```

```

setwd("~/Desktop/MLWBA")
#### Import the data ####
pos = read.csv("POS DATA.csv")
View(pos)

data = pos %>% group_by(Order) %>%
  summarise(Description=paste(Description,collapse = ","))
View(data)

write.csv(data$Description, "big123.csv", quote=FALSE, row.names = FALSE)

#Import file using Apriori algorithm#
big = read.transactions("big123.csv",sep=",")

```

```
## Warning in asMethod(object): removing duplicated items in transactions
```

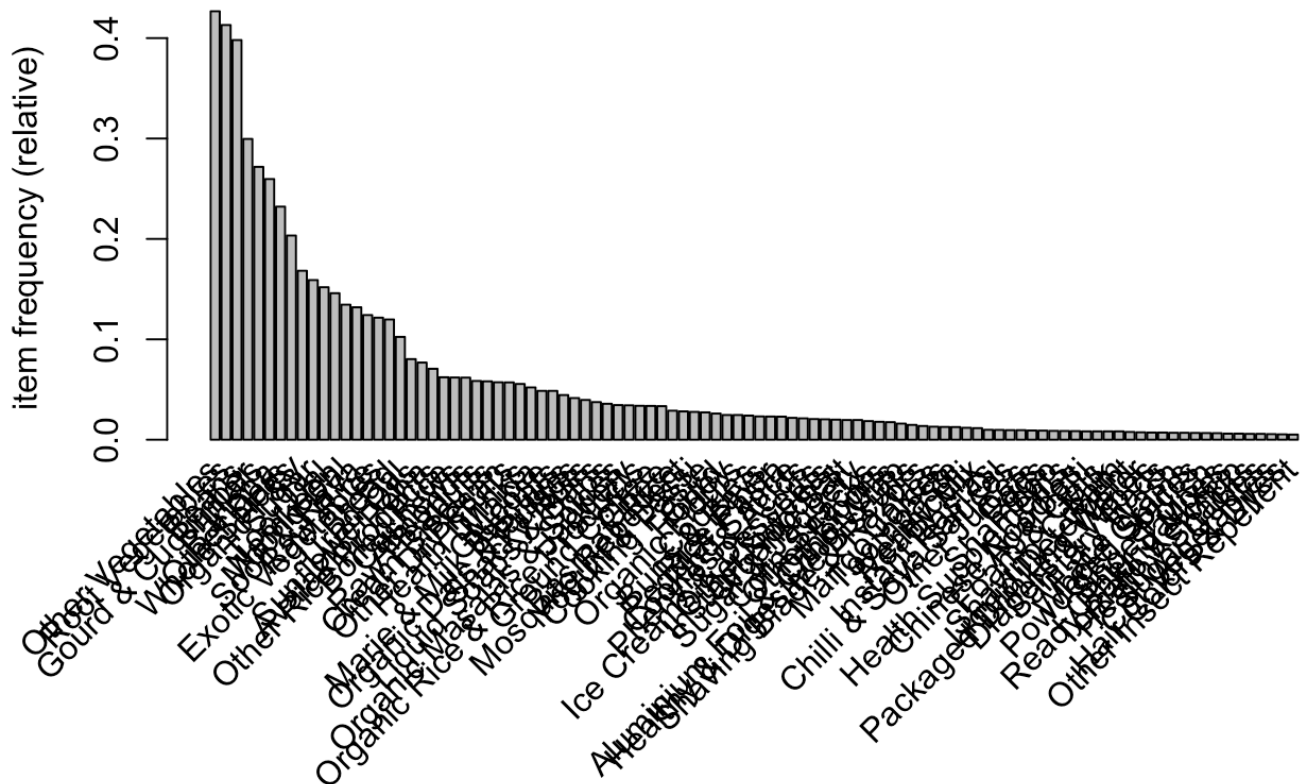
```
summary(big)
```

```

## transactions as itemMatrix in sparse format with
## 8408 rows (elements/itemsets/transactions) and
## 241 columns (items) and a density of 0.02514548
##
## most frequent items:
## Other Vegetables   Root Vegetables           Beans Gourd & Cucumber
##              3588              3473              3347              2519
##           Brinjals           (Other)
##           2285           35741
##
## element (itemset/transaction) length distribution:
## sizes
##      1      2      3      4      5      6      7      8      9     10     11     12     13     14     15     16
## 654   644   886 1039   995   966   797   659   513   305   240   197   145   104    84    57
##     17     18     19     20     21     22     23     24     25     26     28
##     29     22     26     13      9     12      5      1      2      3      1
##
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      1.00    3.00    5.00    6.06    8.00   28.00
##
## includes extended item information - examples:
##      labels
## 1          0
## 2          2
## 3 7831582

```

```
itemFrequencyPlot(big, topN=100)
```



```
rules = apriori(big, parameter = list(supp=0.005,conf=0.0))
```

```
## Apriori
##
## Parameter specification:
## confidence minval smax arem aval originalSupport maxtime support minlen
##          0    0.1   1 none FALSE                TRUE      5   0.005     1
## maxlen target  ext
##      10  rules TRUE
##
## Algorithmic control:
## filter tree heap memopt load sort verbose
##    0.1 TRUE TRUE  FALSE TRUE    2    TRUE
##
## Absolute minimum support count: 42
##
## set item appearances ...[0 item(s)] done [0.00s].
## set transactions ...[241 item(s), 8408 transaction(s)] done [0.00s].
## sorting and recoding items ... [100 item(s)] done [0.00s].
## creating transaction tree ... done [0.00s].
## checking subsets of size 1 2 3 4 5 6 7 done [0.01s].
## writing ... [21474 rule(s)] done [0.00s].
## creating S4 object ... done [0.00s].
```

```
write(rules,file="bigrules.csv",sep=",", row.names=FALSE)
```

```
plot(rules,method = "graph", control = list(type="item"),interactive = T)
```

```
## Warning in plot.rules(rules, method = "graph", control = list(type = "item"), :  
## The parameter interactive is deprecated. Use engine='interactive' instead.
```

```
## Warning: Unknown control parameters: type
```

```
## Available control parameters (with default values):  
## main = Graph for 100 rules  
## max = 100  
## nodeCol = c("#EE0000FF", "#EE0303FF", "#EE0606FF", "#EE0909FF", "#EE0C0CFF",  
"#EE0F0FFF", "#EE1212FF", "#EE1515FF", "#EE1818FF", "#EE1B1BFF", "#EE1E1EFF", "#EE  
2222FF", "#EE2525FF", "#EE2828FF", "#EE2B2BFF", "#EE2E2EFF", "#EE3131FF", "#EE3434  
FF", "#EE3737FF", "#EE3A3AFF", "#EE3D3DFF", "#EE4040FF", "#EE4444FF", "#EE4747FF",  
"#EE4A4AFF", "#EE4D4DFF", "#EE5050FF", "#EE5353FF", "#EE5656FF", "#EE5959FF", "#EE  
5C5CFF", "#EE5F5FFF", "#EE6262FF", "#EE6666FF", "#EE6969FF", "#EE6C6CFF", "#EE6F6F  
FF", "#EE7272FF", "#EE7575FF", "#EE7878FF", "#EE7B7BFF", "#EE7E7EFF", "#EE8181F  
F", "#EE8484FF", "#EE8888FF", "#EE8B8BFF", "#EE8E8EFF", "#EE9191FF", "#EE9494FF",  
"#EE9797FF", "#EE9999FF", "#EE9B9BFF", "#EE9D9DFF", "#EE9F9FFF", "#EEA0A0FF", "#EE  
A2A2FF", "#EEA4A4FF", "#EEA5A5FF", "#EEA7A7FF", "#EEA9A9FF", "#EEABABFF", "#EEACAC  
FF", "#EEAEAEFF", "#EEB0B0FF", "#EEB1B1FF", "#EEB3B3FF", "#EEB5B5FF", "#EEB7B7FF",  
"#EEB8B8FF", "#EEBABAFF", "#EEBCBCFF", "#EEBDBDFF", "#EEBFBFFF", "#EEC1C1FF", "#EE  
C3C3FF", "#EEC4C4FF", "#EEC6C6FF", "#EEC8C8FF", "#EEC9C9FF", "#EECBCBFF", "#EECDC  
DFF", "#EECFCFFF", "#EED0D0FF", "#EED2D2FF", "#EED4D4FF", "#EED5D5FF", "#EED7D7F  
F", "#EED9D9FF", "#EEDBDBFF", "#EEDCDCFF", "#EEDEDEFF", "#EEE0E0FF", "#EEE1E1FF",  
"#EEE3E3FF", "#EEE5E5FF", "#EEE7E7FF", "#EEE8E8FF", "#EEEAEEAFF", "#EEECECFF", "#EE  
EEEEFF")  
## itemnodeCol = #66CC66FF  
## edgeCol = #ABABABFF  
## labelCol = #000000B3  
## measureLabels = FALSE  
## precision = 3  
## arrowSize = 0.5  
## alpha = 0.5  
## cex = 1  
## layout = NULL  
## layoutParams = list()  
## engine = igraph  
## plot = TRUE  
## plot_options = list()  
## verbose = FALSE
```

```
## Warning: Too many rules supplied. Only plotting the best 100 using 'lift'  
## (change control parameter max if needed).
```